Cosmos vial inspection technical report

Project VyFlare | October 31 2026 | Prepared for the project team and technical reviewers

Executive assessment

We built and tested a reversible Cosmos3-Nano LoRA adaptation for visible liquid-color observations, extended the trainer to synthetic images, and implemented the worker interface expected by the recipe-detection backend. The current adapter runs on the DGX Spark. Direct compatibility with the reference backend's actual client has passed live inference checks.

The project is ready for controlled backend integration testing. It is not yet a fully validated factory inspection system. The complete frontend-to-backend-to-Cosmos handoff remains untested, and no independent real-session ingredient, volume, contamination or recipe accuracy has been established. A successful connection is evidence of software compatibility, not manufacturing accuracy.

Area	Current status
Active model	Version 1 step-20 Cosmos3-Nano LoRA adapter
Synthetic continuation	Completed experimentally; not promoted after real final-color regression
Field-data preparation	Audited; provisional observations remain ineligible for training
Backend interface	Implemented and live-tested with the actual reference client
Application deployment	Team backend configuration and full application test still pending
Content screening	Absent in our LoRA worker; explicitly disclosed in responses
Production accuracy and business impact	Not measured

Manufacturing purpose and scope

The intended manufacturing use is to help identify visible fill discrepancies before a vial leaves the line and provide review evidence alongside recipe and process measurements. Lower waste, rework and escaped defects are possible benefits, but none has been measured in this project.

The Physical AI work connects video from physical filling equipment with a vision-language model and evaluates synthetic camera renders from a digital twin. We used the supplied real-line footage and field captures and ran adaptation and inference on the Spark. This report does not claim that software controlled the filling equipment or that simulated liquid behavior reproduces actual mixing.

Current branch: `codex/cosmos3-nano-finetune`. Code baseline for this report: `13b4e58`. Integration reference: `model4/recipe-detection-backend-integration` at `956d5d24d19f80b99f85d4841e678d20d533e0fb`. [E1, E7]

Data used and data deliberately withheld

The original training targets are human-reviewed whole-clip visible-color descriptions from video 5. Earlier model predictions were retained as audit metadata and excluded from supervision. The confirmed source vocabulary treats T200 as red. A source tank's color and a vial's mixed appearance are separate observations; the model was not taught to equate them. [E1]

Source	Evidence and use
Video 5 and verification workbook	30 reviewed clips; 21 training and nine development-validation clips
Correct recipes.xlsx and field-data.zip	Intended recipe bank and 30 provisional vial records, six videos and 30 photos; audited, not used as new training truth
Field photos and videos	Used for exploratory paired inference; folder pairing is not verified physical-vial identity
Digital-twin archive	600 PNGs at 640 x 480; 12 render-color controls and 50 authored frames per control; synthetic color-class experiment
Other organizer recordings and views	Discussed as additional evidence sources; no claim of verified supervision from them in the completed runs

The field audit found 15 conflicts between intended and provisionally observed ingredient order, 12 ingredient durations longer than their complete vial windows, and one record without a unique candidate video-window mapping. These are evidence-quality problems, not proof of process defects. Missing quantities and durations remain missing rather than being converted to zero. All 30 field records retain unverified review status and are excluded from training, as requested. [E2]

Partitioning and leakage controls

The completed real pilot held out all clips from recipes 6, 9 and 13, giving a 21/9 partition. It did not use an 80/20 random frame split. Both sides still share one recording session, so the result is a development benchmark rather than an independent factory test.

For a future 80/20 experiment, assign complete independent recording sessions or production batches to partitions before extracting clips or frames. Keep the same vial, adjacent views and related synthetic variants in one partition. Use an independent validation allocation or training-only cross-validation for tuning; leave the final test partition untouched. With this small dataset, trustworthy grouping matters more than hitting exactly 80 percent.

The digital twin has 565 unique hashes and 35 repeated files. Preparation removes exact duplicate leakage, holds out the last ten authored frame settings, and excludes two intermediate settings as a gap. The scene, camera and lighting are still shared across partitions, which limits the strength of the synthetic benchmark. [E3, E4]

Model adaptation and changes from the pretrained model

We selected the Cosmos3-Nano reasoner pathway for image and video interpretation. The pilot extracts the Qwen3-VL-shaped reasoner from the cached NVIDIA checkpoint using a strict one-to-one, name-and-shape-checked mapping of 750 tensors, including 351 vision tensors. It does not substitute another model's weights. The diffusion pathway is outside this adaptation. [E1]

Setting	Implemented value
Source model	nvidia/Cosmos3-Nano
Source snapshot	e59a53c25979a090fa8706c9acc0c254a6e89b92
Trainable portion	Language-attention q, k, v and output projections through LoRA
LoRA settings	Rank 8; alpha 16; dropout 0.05
Parameters updated	7,667,712; base and vision weights frozen
Training	BF16, SDPA, gradient checkpointing, assistant-answer-only loss
Optimizer	AdamW; learning rate 5e-5; accumulation four; seed 42
Pilot budget	20 optimizer updates
Video input	32 uniform frames; 2,621,440 pixels across the entire sampled video

The total-video pixel budget reduces these 640 x 480 inputs to approximately 320 x 224 per sampled frame. This enabled the bounded experiment on a single Spark, but limits visibility of thin streams, subtle shades and brief changes. Uniform sampling and whole-clip labels do not provide event-boundary supervision.

Real video development results

Metric on nine fixed clips	Base reasoner	Version 1 adapter
Required color-list schema	7/9	9/9
Exact whole-clip color sequence	0/9	4/9 or 44.4 percent
Exact final color label	3/9	5/9 or 55.6 percent

Version 1 improved structured output and label agreement on this small within-session benchmark. Three blue-to-dark-green targets collapsed to green; another clip lost its orange-to-brown transition; one acquired an extra yellow stage. These failures matter because a concise final color can conceal a process change. The percentages are not chemical-identification accuracy and cannot be generalized to new sessions. [E1]

The base checkpoint is preserved. Versioned adapters and hashes allow rollback and future comparison. Additional fine-tuning is technically straightforward; trustworthy labels and independent evaluation are the harder requirements. Corrected labels should normally trigger a fresh comparison from the same base rather than assuming continued training removes earlier mistakes.

Synthetic adaptation and what changed in the second experiment

The trainer now accepts native still images as well as videos, with EXIF/RGB handling and a separate 262,144-pixel image budget. Image targets use render_color_class, while real videos retain observed_color_sequence. This keeps renderer labels separate from human observations. Continuation loading verifies the parent model and adapter hash and starts a fresh optimizer; it is not a full optimizer-state resume. [E4]

The prepared set contains 427 synthetic training images plus 21 reviewed videos, and 120 synthetic validation images plus nine real regression clips. Fifty-three synthetic cases were excluded through the combined split-gap and duplicate rules. The bounded continuation presented only 40 images and 40 videos over 20 updates; it did not train a complete epoch over all prepared records.

Evaluation used 12 synthetic holdout images at one frame setting and all nine real regression clips. The other 108 prepared synthetic validation images were not scored.

Metric on matched inputs	Base	Version 1	Continued adapter
Synthetic class with required JSON	8/12	0/12	9/12
Synthetic class accepting bare words	8/12	8/12	9/12
Synthetic schema validity	12/12	0/12	12/12
Real exact color sequence	0/9	4/9	4/9
Real exact final color	3/9	5/9	4/9

Version 1's zero strict synthetic score was a format effect: it returned bare color words for the new task. The visual class gain when accepting bare words was one case. The continued adapter restored the required synthetic format but reduced real final-color agreement by one case. We therefore retained Version 1 for serving instead of promoting the continuation. [E4]

Digital twin value and limitations

The twin supplies controlled labels cheaply and can support future variation in camera pose, lighting, materials and fill appearance. The supplied corpus currently varies render-color control and authored animation frame within one fixed scene. Its labels are not chemical identities, measured proportions, defects or calibrated volumes.

Because the liquid animation does not model physical mixing or cross contamination, these renders cannot establish accuracy for those phenomena. An unrealistic image can still teach useful features; transfer must be measured on independent real footage. A matched-budget real-only versus mixed-training experiment is needed to isolate synthetic data's contribution. The present experiment does not prove a factory benefit or that synthetic data is inherently ineffective. [E3, E4]

DGX Spark usage and engineering issues

Recorded experiments used one DGX Spark with an NVIDIA GB10 GPU and approximately 121 GiB of reported host unified memory. The recorded training runtime was Python 3.13.16, PyTorch 2.10.0+cu130, Transformers 4.57.6 and PEFT 0.18.1. Peak allocated CUDA memory is an allocator measurement, not the total memory footprint of all system workloads. [E1, E4]

Recorded activity	Job wall time	Optimizer loop	Peak CUDA allocation
One-update smoke run	136.17 seconds	4.09 seconds for the update	19.44 GiB
Version 1 full pilot	487.17 seconds or 8.12 minutes	268.67 seconds	19.54 GiB
Synthetic continuation	479.41 seconds or 7.99 minutes	179.61 seconds	19.54 GiB

The two full 20-update jobs total 966.58 seconds, or 16.11 minutes. Including the smoke job gives approximately 18.38 minutes of recorded job wall time. These intervals include loading and evaluations. They are not exclusive GPU time, a complete Spark utilization ledger, energy consumption or a billed-cost measurement. Always-on serving, additional inference, preflight, data transfer and preparation are outside that total.

The recorded six-video/six-photo paired field diagnostic used 88.65 seconds of summed server processing time. The initial 12-image synthetic diagnostic used 56.98 seconds. These include preprocessing and both model predictions and should not be interpreted as pure GPU compute. Backend integration and branch cleanup ran zero training updates. [E3, E5]

Engineering issues included strict checkpoint-name mapping, the distinction between total-video and per-frame pixel budgets, and maintaining assistant-only loss boundaries. PyTorch warned about GB10 compute capability, but inference, backward passes and optimizer updates succeeded in the recorded environment. That success does not guarantee every package combination works.

Windows sandbox permissions initially prevented temporary test-file creation; approved reruns passed. The intentional runtime-failure fixture produces a logged traceback while testing the 500 response. That log is expected test behavior, not a failed training run.

New field inference findings

Twelve paired requests compared base and Version 1 on six fixed video windows and six folder-selected photos. Version 1 produced the required schema on 6/6 videos versus 5/6 for the base, but 0/6 photos versus 1/6 for the base. Five adapter video answers collapsed to one color. Photo format failures do not by themselves establish wrong visual colors. Since the field labels and same-vial pairings remain unverified, no defensible field accuracy can be reported. [E5]

Backend integration implementation and cleanup

The sole integration target is model4/recipe-detection-backend-integration at commit 956d5d2. We replaced the earlier custom evidence-gateway direction with the request/response interface used by its actual RemoteCosmosModel client. The branch retains Cosmos training and serving code; the reference backend retains its measurement models and decision policy. [E6, E7]

Interface element	Current implementation
Worker route	POST /backend/predict on port 7865
Multipart request	image, frame_id, model and prompt
Model names	cosmos or cosmos_lite in worker requests
Response	Echoed identifiers, mode=real, inference_ms and result.text
Provenance	Actual Linux/aarch64/NVIDIA GB10 execution metadata, adapter identity, applied prompt and image hash
Input validation	JPEG/PNG; request at most 5 MiB; image at most 16 megapixels
Concurrency	Same nonblocking gate as the existing browser tester; busy requests return 429

Caller prompts now reach inference. Temporary decoded images are deleted, and inference errors release the shared gate. The browser tester still supports the original video/photo route. Our worker remains a single resident model rather than a new model per request.

Cleanup removed three obsolete gateway/client modules, their context configuration, 13 gateway tests, two deployment guides and the consolidated PDF describing the retired approach. Dependencies are now named requirements-worker.txt. Current tracked contents were checked for retired integration references; Git history remains available for auditability. This report replaces that removed consolidated report without restoring the retired implementation.

Compatibility evidence and its boundary

All 32 remaining tests passed, including five direct-worker contract tests. The earlier total of 45 included the 13 removed gateway tests. The actual reference client passed a fixture-based contract check and live inference against the Spark; a later readiness recheck passed again. Fixture hardware metadata was not presented as proof of real GPU execution. [E6, E8]

A real 640 x 480 end frame from vial-04 produced the answer that the visible liquid was black. The region looked very dark with greenish edges. The complete clip through the original video route returned valid green color-list JSON in 3.23 seconds. This illustrates sensitivity to framing, context and prompt; it is not a scored accuracy comparison.

The reference branch defaults to a native Cosmos Framework worker on 8001. Our extracted reasoner and Version 1 LoRA on 7865 share the Cosmos3-Nano source family but are not identical in weights, processing or screening. The native worker was left unchanged. The full team's frontend/backend flow has not yet been exercised with our worker. [E6, E8]

Guardrail screening assessment

The reference native worker explicitly screens input text, uploaded visual content and generated text, with Framework visual postprocessing. Our LoRA worker does not load those screening components and reports that absence in result.safety. Backend compatibility tests accept our envelope, but do not establish identical screening behavior. [E7]

NVIDIA's Framework documents text and visual content-safety filters and face-related postprocessing. These address content safety; they do not establish the correct vial color, ingredient source, delivered volume, mixing physics or recipe compliance. Adding content screening would not by itself improve our inspection accuracy. [E9]

Controlled internal testing can proceed with trusted footage while the absence is documented. Before expanding unrestricted uploads or claiming parity with the native worker, implement and test suitable screening or integrate through the native screened path. This is a deployment recommendation, not a claim that the exact native screening is a protocol requirement. No screening integration was implemented during the discussion.

Acceptance work before a complete deployment

1. Configure the team's running backend to reach the worker and exercise frontend submission, prompt forwarding, response display, busy/error handling and reconnection. Confirm that Cosmos remains advisory and existing Model 4 review requirements are preserved.

2. Establish verified vial identity, station/camera alignment and dispense-event timestamps. Controller activation logs are stronger source-identity evidence than expected recipe order.

3. Calibrate actual per-fluid delivery with measured volume or mass. Intended recipe quantities must not be treated as measured dispense labels.

4. Freeze independent real-session test data across lighting, fill states, mixtures and recipe transitions. Score appearance, ingredient identity, timing, quantity and known/unknown recipe decisions separately where verified labels exist.

5. Run matched-budget real-only and mixed-synthetic adaptations against the same untouched real test set. Promote a candidate only after a documented comparison and review of regressions.

The system currently lacks validated getRecipe orchestration and a calibrated recipe-match percentage from this Cosmos adapter. Backend recipe lookup and explicit comparison logic should use trustworthy observations and tolerances; model prose is not a probability calibration.

Retraining is supported through new versioned datasets and adapter output directories. Continuing an existing adapter uses a fresh optimizer; starting from the unchanged base provides a cleaner comparison when correcting labels. Preserve source hashes, old adapters, fixed evaluation cases and replay data to detect regressions.

Reproducibility and handoff

The active adapter SHA-256 is:

`9dae56b981de7e7de200e39580e57cbcba81d3efdcf940f2e025e80a41a3a5f4`

The unpromoted synthetic continuation SHA-256 is:

`792345ea39e4118c1f31907f38221ae3265b8cfb0c6480a12b5407d099d89a3e`

The Spark experiment root is /home/yt-vyflare71b7/workspace/vial-finetune-20261029. The serving adapter is runs/pilot-v1/adapter-step-020. Updated worker scripts are in scripts/model4-worker-v1. Private datasets, checkpoints and run evidence are ignored by Git; distributing this source branch alone does not distribute a runnable checkpoint bundle. Preserve the model and data records separately.

When the backend runs on the Spark, configure COSMOS_FULL_URL to http://127.0.0.1:7865/backend/predict. From another host, use an SSH tunnel or a reachable private endpoint. The frontend model selection remains cosmos_full; the remote worker name remains cosmos. Detailed tunnel and environment settings are in reports/model4-backend-integration-guide.md. No change to the team's live backend configuration is claimed here.

Evidence index

• E1: reports/spark-pilot-v1.md; outputs/spark-pilot-v1/run.json, smoke-run.json and training.jsonl. Original training method, metrics, model provenance and resource readings.

• E2: reports/field-review-v1.md; data/local/field-review-v1/audit.json. Field inventory, recipe/observation separation, conflicts and training exclusion.

• E3: reports/synthetic-diagnostic-v1.md. Archive verification, fixed-scene limitations and 12-image inference diagnostic.

• E4: reports/synthetic-training-v1.md; outputs/synthetic-mixed-v1/run.json and artifact-verification.json. Mixed-data exposure, regression and promotion decision.

• E5: reports/field-diagnostic-v1.md. Twelve paired field requests and the distinction between schema compliance and visual accuracy.

• E6: reports/model4-integration-correction.md; scripts/model4_worker_contract.py; tests/test_model4_worker_contract.py. Interface changes, cleanup and test scope.

• E7: Reference backend implementation at commit 956d5d2. Remote client contract and native worker screening implementation.

• E8: outputs/model4-backend-contract/client-live-test.json, client-cleanup-check.json, client-readiness-recheck.json and legacy-video-smoke.json. Direct client and live inference evidence, not full application acceptance.

• E9: NVIDIA Cosmos Framework inference documentation, Guardrails section. Reviewed October 31 2026; external documentation may change.

Code changes retained for review: 5042951 implements the backend-compatible worker; 13b4e58 removes the retired integration. This report records completed observations and explicitly identifies recommendations and unverified claims.